MTM4692

Spring 2026

Fettah KIRAN
Department of AI and Data Engineering
fkiran@yildiz.edu.tr

Common Table Expressions (CTEs)

WITH · Recursive CTEs · Hierarchical Queries

📋 Today's Agenda (1:00 PM – 3:45 PM)

Time	Topic	Type
1:00 – 1:15	Week 6 recap poll	Review
1:15 – 1:50	Common Table Expressions (WITH)	**Theory** 📖
1:50 – 2:15	Advanced CTE Patterns	**Theory** 📖
2:15 – 2:30	☕ Break (15 min)	
2:30 – 2:55	Recursive CTEs & Hierarchies	**Theory** 📖
2:55 – 3:10	CTE Performance & Best Practices	**Theory** 📖
3:10 – 3:40	Hands-on: Complex Queries with CTEs	**Practice** 💻
3:40 – 3:45	Recap & next week preview	Wrap-up

Week 6 Quick Recap

Concept	Key Point
ACID	Atomicity, Consistency, Isolation, Durability — relational guarantees
BASE	Basically Available, Soft state, Eventual consistency
CAP Theorem	Distributed: pick 2 of C, A, P (partitions inevitable)
Document DB	JSON documents, flexible schema (MongoDB)
Key-Value	Simple key→value, ultra-fast (Redis)
Graph DB	Nodes + edges for relationship queries (Neo4j)
Polyglot persistence	Use the right DB type for each data need
SQLite JSON	`json_extract()`, `json_set()`, `json_each()`

Common Table Expressions (CTEs)

Tutorial link: https://www.geeksforgeeks.org/sql/cte-in-sql/

What Is a CTE?

A (CTE) is a **named temporary result set** defined within a query using `WITH`.

```sql
-- WITHOUT CTE (nested subquery - hard to read)
SELECT dept_name, avg_gpa
FROM (
    SELECT d.dept_name, AVG(s.gpa) AS avg_gpa
    FROM student s
    JOIN department d ON s.dept_id = d.dept_id
    GROUP BY d.dept_id
) sub
WHERE avg_gpa > 3.0;

-- WITH CTE (much cleaner!)
WITH dept_averages AS (
    SELECT d.dept_name, AVG(s.gpa) AS avg_gpa
    FROM student s
    JOIN department d ON s.dept_id = d.dept_id
    GROUP BY d.dept_id
)
SELECT dept_name, avg_gpa
FROM dept_averages
WHERE avg_gpa > 3.0;
```

CTE Syntax & Multiple CTEs

```sql
-- Single CTE
WITH cte_name AS (
    SELECT ...
)
SELECT ... FROM cte_name ...;

-- Multiple CTEs (comma-separated)
WITH
    high_gpa AS (
        SELECT * FROM student WHERE gpa >= 3.5
    ),
    cs_courses AS (
        SELECT * FROM course WHERE dept_id = 1
    ),
    enrollments AS (
        SELECT e.* FROM enrollment e
        JOIN high_gpa h ON e.student_id = h.student_id
        JOIN cs_courses c ON e.course_id = c.course_id
    )
SELECT * FROM enrollments;
> 💡 Each CTE can reference previously defined CTEs in the same WITH block.
```

CTE vs Subquery vs View

Feature	Subquery	CTE	View
Scope	Single query	Single query	Entire database
Reusable?	No	Yes (within query)	Yes (across queries)
Stored?	No	No	Yes (definition)
Readability	Can be messy nested	Clean, top-down	Separate object
Recursive?	No	Yes!	No
Performance	Same as CTE	Same as subquery	Same (re-executed)

> 💡 Use **CTE** when you need to reference a subquery multiple times in one query, or when you need recursion.

🧠 Exam Focus: CTEs

Q1: What is the main advantage of a CTE over a subquery?

A) CTEs are always faster

B) **CTEs improve readability and can reference themselves recursively** ✅

C) CTEs store data permanently

D) CTEs can replace tables

Q2: How many times can you reference a CTE within its associated query?

A) Only once

B) **Multiple times** ✅

C) Exactly twice

D) It depends on the database

Recursive CTEs

A CTE that **references itself** — used for hierarchical/tree data.

```sql
-- Syntax:
WITH RECURSIVE cte_name AS (
    -- Base case (anchor member)
    SELECT ...
    UNION ALL
    -- Recursive case (references cte_name)
    SELECT ... FROM cte_name WHERE ...
)
SELECT * FROM cte_name;
```

Two required parts:

1. **Anchor member**: Initial query (base case) — no self-reference
2. **Recursive member**: References the CTE itself — builds on previous results

Recursive CTE: Number Series

```sql
-- Generate numbers 1 to 10
WITH RECURSIVE numbers AS (
    -- Base case
    SELECT 1 AS n
    UNION ALL
    -- Recursive case
    SELECT n + 1 FROM numbers WHERE n < 10
)
SELECT n FROM numbers;
```

Execution trace:

```
Step 1 (base):      n = 1
Step 2 (recursive): n = 2  (1 + 1, since 1 < 10)
Step 3 (recursive): n = 3  (2 + 1, since 2 < 10)
...
Step 10 (recursive): n = 10 (9 + 1, since 9 < 10)
Step 11: 10 < 10 is FALSE → stop
```

Recursive CTE: Date Series

```sql
-- Generate all dates in January 2025
WITH RECURSIVE dates AS (
    SELECT date('2025-01-01') AS d
    UNION ALL
    SELECT date(d, '+1 day') FROM dates
    WHERE d < '2025-01-31'
)
SELECT d AS date FROM dates;
```

> 💡 Very useful for filling gaps in time-series data or generating calendar tables.

Recursive CTE: Employee Hierarchy

```sql
-- Employee → Manager hierarchy
CREATE TABLE employee (
    emp_id INTEGER PRIMARY KEY,
    name TEXT,
    manager_id INTEGER REFERENCES employee(emp_id)
);

INSERT INTO employee VALUES
(1, 'CEO', NULL),
(2, 'VP Engineering', 1),
(3, 'VP Sales', 1),
(4, 'Dev Manager', 2),
(5, 'Sales Manager', 3),
(6, 'Developer', 4),
(7, 'Developer', 4),
(8, 'Sales Rep', 5);

-- Traverse the hierarchy
WITH RECURSIVE org_chart AS (
    SELECT emp_id, name, manager_id, 0 AS level,
           name AS path
    FROM employee WHERE manager_id IS NULL
    UNION ALL
    SELECT e.emp_id, e.name, e.manager_id, oc.level + 1,
           oc.path || ' > ' || e.name
    FROM employee e
    JOIN org_chart oc ON e.manager_id = oc.emp_id
)
SELECT level, SUBSTR('        ', 1, level * 4) || name AS hierarchy
FROM org_chart ORDER BY path;
```

Recursive CTE Output

```
level | hierarchy
------+----------------------------
  0   | CEO
  1   |     VP Engineering
  2   |         Dev Manager
  3   |             Developer
  3   |             Developer
  1   |     VP Sales
  2   |         Sales Manager
  3   |             Sales Rep
```

> 💡 The `level` column tracks depth. The `path` column tracks the full chain.

🧠 Exam Focus: Recursive CTEs

Q3: What are the two required parts of a recursive CTE?

A) SELECT and WHERE clauses

B) **Anchor member (base case) and recursive member** ✅

C) PARTITION BY and ORDER BY

D) INSERT and SELECT statements

Q4: What operation connects the anchor and recursive members in a recursive CTE?

A) JOIN

B) INTERSECT

C) **UNION ALL** ✅

D) CROSS JOIN

CODING PRACTICE

Common Table Expressions (CTEs)

Practice: CTEs

```sql
-- CTE: department averages above 3.0
WITH dept_avg AS (
    SELECT d.dept_name,
           ROUND(AVG(s.gpa), 2) AS avg_gpa,
           COUNT(*) AS student_count
    FROM student s
    JOIN department d ON s.dept_id = d.dept_id
    GROUP BY d.dept_id, d.dept_name
)
SELECT * FROM dept_avg WHERE avg_gpa > 3.0 ORDER BY avg_gpa DESC;

-- Multiple CTEs: students above their dept average
WITH dept_avg AS (
    SELECT dept_id, AVG(gpa) AS avg_gpa
    FROM student GROUP BY dept_id
),
above_avg AS (
    SELECT s.first_name, s.last_name, s.dept_id, s.gpa,
           ROUND(da.avg_gpa, 2) AS dept_avg
    FROM student s
    JOIN dept_avg da ON s.dept_id = da.dept_id
    WHERE s.gpa > da.avg_gpa
)
SELECT * FROM above_avg ORDER BY dept_id, gpa DESC;
```

💻 Practice: Recursive CTE — Number Series

```sql
-- Generate numbers 1 to 20
WITH RECURSIVE nums AS (
    SELECT 1 AS n
    UNION ALL
    SELECT n + 1 FROM nums WHERE n < 20
)
SELECT n FROM nums;

-- Fibonacci sequence
WITH RECURSIVE fib AS (
    SELECT 1 AS n, 0 AS fib_n, 1 AS fib_n1
    UNION ALL
    SELECT n + 1, fib_n1, fib_n + fib_n1
    FROM fib WHERE n < 15
)
SELECT n, fib_n AS fibonacci FROM fib;

-- Date series for a month
WITH RECURSIVE dates AS (
    SELECT date('2025-01-01') AS d
    UNION ALL
    SELECT date(d, '+1 day') FROM dates WHERE d < '2025-01-31'
)
SELECT d, strftime('%w', d) AS day_of_week,
       CASE CAST(strftime('%w', d) AS INTEGER)
           WHEN 0 THEN 'Sunday' WHEN 1 THEN 'Monday'
           WHEN 2 THEN 'Tuesday' WHEN 3 THEN 'Wednesday'
           WHEN 4 THEN 'Thursday' WHEN 5 THEN 'Friday'
           WHEN 6 THEN 'Saturday'
       END AS day_name
FROM dates;
```

💻 Practice: Recursive CTE — Hierarchy

```sql
-- Create employee hierarchy table
CREATE TABLE IF NOT EXISTS emp_hierarchy (
    emp_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    title TEXT,
    manager_id INTEGER REFERENCES emp_hierarchy(emp_id)
);

DELETE FROM emp_hierarchy;
INSERT INTO emp_hierarchy VALUES
(1, 'Ayşe', 'CEO', NULL),
(2, 'Mehmet', 'CTO', 1),
(3, 'Fatma', 'CFO', 1),
(4, 'Ali', 'Dev Lead', 2),
(5, 'Zeynep', 'Accountant', 3),
(6, 'Emre', 'Developer', 4),
(7, 'Selin', 'Developer', 4),
(8, 'Can', 'Intern', 6);

-- Full org chart
WITH RECURSIVE org AS (
    SELECT emp_id, name, title, manager_id,
           0 AS depth, name AS chain
    FROM emp_hierarchy WHERE manager_id IS NULL
    UNION ALL
    SELECT e.emp_id, e.name, e.title, e.manager_id,
           o.depth + 1, o.chain || ' → ' || e.name
    FROM emp_hierarchy e JOIN org o ON e.manager_id = o.emp_id
)
SELECT depth,
       SUBSTR('                ', 1, depth * 4) || name
           || ' (' || title || ')' AS org_chart,
       chain
FROM org ORDER BY chain;
```

Summary: Common Table Expressions (CTEs)

Concept	Purpose	Use Case
`WITH ... AS`	Named temporary subquery	Readable, modular queries
`Multiple CTEs`	Chain dependent CTEs	Complex multi-step analysis
`WITH RECURSIVE`	Self-referencing CTE	Hierarchies, sequences, date series
Anchor member	Base case of recursion	Starting point for recursion
Recursive member	Recursive rule	Builds on previous results
`UNION ALL`	CTE combiner	Required for recursive CTEs

Practice Exercises (Homework)

1. **CTE exercise:**
   - Create a CTE for departments with average GPA > 3.0
   - Use multiple CTEs to find students above their department average
   - Combine CTEs with filtering and sorting
2. **Recursive CTE:**
   - Generate all dates for the current month
   - Build an org chart with depth levels and indentation
   - Calculate factorial numbers using recursive CTE

Next Week Preview

Week 8: Midterm Review

Topics covered on the midterm:

- Database concepts, relational model, keys
- SQL DDL/DML/DQL, data types, constraints
- JOINs, GROUP BY, HAVING, subqueries
- Indexes, EXPLAIN, B-Tree
- Views, WITH CHECK OPTION
- ACID vs BASE, CAP theorem, NoSQL types
- Common Table Expressions (CTEs), recursive patterns

MIDTERM REVIEW: Weeks 1–7 Summary

Week 1: Introduction to Databases

Core Concepts:

- **Relational Model**: Tables with rows (tuples) and columns (attributes)
- **Primary Key (PK)**: Uniquely identifies each row
- **Foreign Key (FK)**: Links to another table's primary key
- **Entity Relationship Diagram (ERD)**: Visualizes table relationships
- **Cardinality**: 1:1, 1:M, M:M relationships
- **Database Instances**: Collection of tables at a specific time

Key Takeaway:

> Design tables with proper keys and relationships to ensure data integrity and efficient queries.

Week 2: SQL Fundamentals

Data Definition Language (DDL):

```
CREATE TABLE, ALTER TABLE, DROP TABLE, TRUNCATE
```

Data Manipulation Language (DML):

```
INSERT INTO, UPDATE, DELETE
```

Data Query Language (DQL):

```
SELECT - the foundation of all queries
```

Week 3: Advanced Queries

JOIN Types:

- **INNER JOIN**: Only matching rows from both tables
- **LEFT JOIN**: All rows from left table + matches from right
- **RIGHT JOIN**: All rows from right table + matches from left
- **CROSS JOIN**: Cartesian product (all combinations)
- **Self-JOIN**: Join table to itself

Week 3: Advanced Queries — JOINs & Subqueries

Subqueries:

- **Scalar subquery**: Returns single value for WHERE/SELECT
- **Row subquery**: Returns one row with multiple columns
- **Table subquery**: Returns multiple rows/columns
- **Correlated subquery**: References outer query rows

Key Takeaway:

> Master JOINs for combining data; use subqueries for complex filtering and multi-step logic.

Week 4: Indexes & Performance

Index Types:

```
COUNT(*), SUM(), AVG(), MIN(), MAX()
```

GROUP BY:

- Collapses rows into groups
- Aggregate functions compute per group
- Works with single or multiple columns

HAVING Clause:

- Filter groups after GROUP BY (not WHERE)
- Uses aggregate results

Week 4: Aggregation & Grouping

Key Syntax:

```sql
SELECT column, COUNT(*) AS count
FROM table
WHERE condition
GROUP BY column
HAVING COUNT(*) > 10
ORDER BY count DESC;
```

Week 5: Performance & Indexing

Database Indexes:

- **B-Tree**: Most common, ordered structure, supports range queries
- **Hash Index**: Fast exact lookups, no range support
- **Create**: `CREATE INDEX idx_name ON table(column)`

EXPLAIN:

- Shows query execution plan
- Identifies missing indexes and full table scans
- Helps optimize slow queries

Week 5: Performance & Indexing

Key Concepts:

- **Primary Key**: Automatically indexed
- **Foreign Key**: Index recommended for joins
- **Index Trade-off**: Faster reads, slower writes

Rule of Thumb:

> Index columns used in WHERE, JOIN ON, ORDER BY, and GROUP BY

Week 6: Relational vs NoSQL

ACID Guarantee (Relational):

- Virtual table based on SELECT query
- Simplify complex queries
- Hide sensitive columns
- Provide consistent interface

View Types:

- **Simple View**: Single table, no aggregates
- **Complex View**: Multiple tables, joins, aggregates, DISTINCT
- **WITH CHECK OPTION**: Enforce INSERT/UPDATE constraints

Week 6: Relational vs NoSQL

Materialized Views:

- Stores results physically
- Faster but must be refreshed
- Trade-off: storage vs. performance

Key Benefit:

> Reuse complex logic, improve security, simplify maintenance

Week 6: Database Comparison — ACID vs BASE

ACID (Relational Databases):

- **Atomicity**: All or nothing transactions
- **Consistency**: Data remains valid
- **Isolation**: Transactions don't interfere
- **Durability**: Committed data persists

BASE (NoSQL Databases):

- **Basically Available**: System always responds
- **Soft state**: Data may change
- **Eventually Consistent**: Convergence over time

Week 6: Database Comparison — CAP

CAP Theorem:

Pick 2 of 3 in distributed systems:

- **Consistency**: All nodes see same data
- **Availability**: System always responsive
- **Partition tolerance**: Works with network splits

Database Types:

- **Relational (SQL)**: ACID, structured schema
- **Document (MongoDB)**: Flexible JSON, semi-ACID
- **Key-Value (Redis)**: Ultra-fast, simple data
- **Graph (Neo4j)**: Relationship queries

Week 7: CTEs — Common Table Expressions

CTE Benefits:

- Improves readability (top-down flow)
- Reuse same CTE multiple times
- Easier to debug complex queries
- Foundation for recursive queries

CTE vs Alternatives:

Aspect	Subquery	CTE	View
Reuse in query	Once	Multiple	Database-wide
Recursive	No	**Yes**	No
Readable	Nested mess	Clean	Separate

Week 7: Recursive CTEs

Common Uses:

1. **Org Charts**: Employee → Manager hierarchies
2. **Product Categories**: Parent → Child categories
3. **Number Series**: Generate sequences (1 to N)
4. **Date Series**: Fill calendar gaps
5. **Graph Traversal**: Follow relationships

Week 1—7: Key Patterns & Best Practices

Problem-Solving Strategy:

1. **Understand the question**: What tables? What filters?
2. **Start simple**: Begin with basic SELECT
3. **Add complexity**: Layer in JOINs, subqueries, aggregates
4. **Test incrementally**: Verify each step
5. **Optimize if slow**: Check EXPLAIN, add indexes

Data Quality Checks:

- `SELECT COUNT(*) FROM table;` — Verify row count
- `SELECT * FROM table WHERE id IS NULL;` — Find nulls
- `SELECT * FROM table LIMIT 5;` — Preview data

Comprehensive Topic Map — Weeks 1–7

1	**Introduction**	PK, FK, Cardinality, Normalization, ERD
2	**SQL Fundamentals**	DDL, DML, DQL, Constraints, Data Types
3	**Advanced Queries**	JOINs (all types), Subqueries, GROUP BY, Aggregation
4	**Indexes & Performance**	B-Tree, EXPLAIN, Query Optimization
5	**Views**	CREATE VIEW, WITH CHECK OPTION, Access Control
6	**Relational vs NoSQL**	ACID, BASE, CAP Theorem, Database Types
7	**CTEs & Hierarchies**	Common Table Expressions, RECURSIVE

Exam Scope:

✅ Analyze query performance
✅ Compare database approaches
✅ Solve recursive/complex queries

📋 SQL PROJECT — Two-Phase Submission

All work must be tracked in GitHub with regular commits

Phase 1: MILESTONE (After Midterm) — 10%

Share and justify your database idea in a technical report (on GitHub)

Phase 2: FINAL PROJECT (End of Semester) — 15%

Submit complete, runnable SQL code on GitHub that solves a real-life problem

📋 MILESTONE — Project Proposal & Justification

Milestone Deliverables:

1. **GitHub Repository** — Create a public repository for your project
   - README.md explaining the project idea and purpose
   - Meaningful commit messages as you work
2. **Technical Report** — 2-3 pages covering:
   - **Problem Statement**: What real-world problem will your database solve?
   - **Why It Matters**: Justify the importance of your project
   - **Preliminary Schema**: 5-8 tables with relationships (describe or draw ERD)
   - **Proposed SQL Features**: List the SQL techniques you plan to use
3. **Schema Sketch** — ERD or table descriptions (in GitHub repo)
4. **2-3 Sample Queries** — Pseudocode or SQL showing your approach

Milestone Note:

This is your PROJECT PROPOSAL. Full implementation comes in the Final Project submission.

Grading: 10 points total

- Problem & Justification clarity: 4 pts
- Schema design quality: 3 pts
- Feasibility & scope: 3 pts

⭐ GRADING RUBRIC — Milestone (10 points)

Category 1: Problem & Justification (4 points)

Score	Criteria
4	Clear problem; strong justification for importance; realistic scope
3	Problem clear; good justification; minor issues
2	Problem present; adequate justification
1	Problem vague or weak justification

What we're looking for:

- **Real-world Problem**: Does this solve an actual business need?
- **Why It Matters**: Justification for your project choice
- **Feasibility**: Realistic scope (5-8 tables, can implement by end of semester)

Category 2: Schema Design (3 points)

Score	Criteria
3	Schema well-designed; tables appropriate; relationships clear
2	Schema mostly good; minor design issues
1	Schema basic; some design concerns
0	Poor schema or incomplete design

What we're looking for:

- 5-8 tables with clear purpose
- Proper relationships (PK/FK)
- Appropriate data types
- Clear ERD or description

Category 3: Proposed SQL Features (3 points)

Score	Criteria
3	Proposes multiple SQL features; includes Views, Joins, Aggregation
2	Proposes several features; covers main concepts
1	Proposes basic features only
0	No feature planning

What we're looking for:

- Sample queries showing your approach
- Features you plan to implement (Views, CTEs, subqueries, etc.)
- Clear understanding of how to solve the problem

FINAL PROJECT (15 points)

Component	Points
Schema Design & Normalization	3
SQL Implementation & Data	3
Views & Advanced Queries	5
Documentation & Performance	4
TOTAL	**15**

Grading:

Milestone: 10 %

Final Project: 15 %

🚀 Project Ideas (Get Inspired!)

1. **E-Commerce**: Products, Orders, Customers, Inventory
2. **University**: Students, Courses, Enrollment, Grades
3. **Hospital**: Patients, Doctors, Appointments, Medications
4. **Library**: Books, Members, Loans, Returns
5. **Airline**: Flights, Passengers, Bookings, Aircraft
6. **Restaurant**: Menu, Orders, Customers, Reservations

Minimal Scope (Safe Choice):

- 6-8 tables
- 5-7 key queries
- 3 views

Milestone Submission Checklist

- [ ] GitHub repository created (public) and shared link provided
- [ ] README.md in repository describing project idea
- [ ] Technical report (2-3 pages, PDF) uploaded to GitHub
- [ ] Problem statement clearly stated
- [ ] Justification for why project is important
- [ ] Schema design (ERD or table descriptions) in repo
- [ ] List of SQL features you plan to use
- [ ] 2-3 sample queries (pseudocode or SQL)
- [ ] At least 3 meaningful commits showing progress
- [ ] Repository organized and well-documented